```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder,LabelBinarizer
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split,KFold
from sklearn.tree import DecisionTreeClassifier,plot_tree
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report,auc,roc_curve
```

Car Evaluation Dataset

- The Car Evaluation Dataset provides insights into what factors influence car acceptability. By exploring the relationships between factors like safety rating, buying price, maintenance cost, and capacity, we can gain a better understanding of consumer preferences and what factors affect acceptability.
- Number of Instances: **1728** (instances completely cover the attribute space)
- Number of Attributes : **6**
  - buying : v-high,high,med,low
  - maint : v-high,high,med,low
  - doors : 2,3,4,5-more
  - persons : 2,4,more
  - lug_boot : small,med,big
  - safety : low,med,high
  - class : unacc,acc,goot,v-good

```
car = pd.read_csv('/content/drive/MyDrive/DM/datasets/car_evaluation/car.data')
car.head()
```

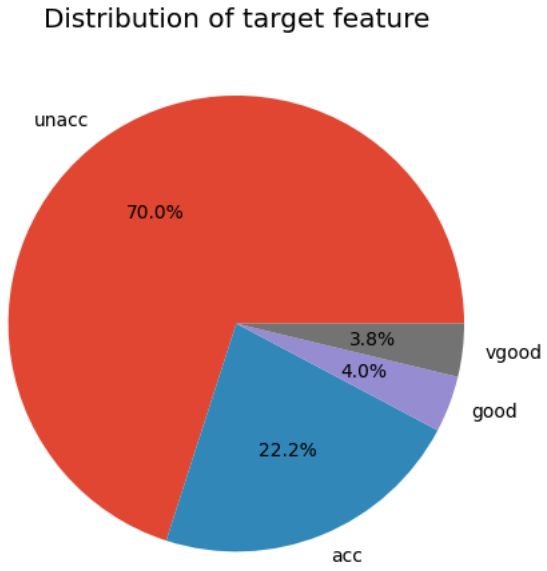
	buying	maint	doors	persons	lug_boot	safety	class
0	vhigh	vhigh	2	2	small	low	unacc
1	vhigh	vhigh	2	2	small	med	unacc
2	vhigh	vhigh	2	2	small	high	unacc
3	vhigh	vhigh	2	2	med	low	unacc
4	vhigh	vhigh	2	2	med	med	unacc

Additional Information about the dataset

```
car.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1728 entries, 0 to 1727
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   buying    1728 non-null   object
 1   maint     1728 non-null   object
 2   doors     1728 non-null   object
 3   persons   1728 non-null   object
 4   lug_boot  1728 non-null   object
 5   safety    1728 non-null   object
 6   class     1728 non-null   object
dtypes: object(7)
memory usage: 94.6+ KB
```

```
plt.style.use("ggplot")
plt.title("Distribution of target feature")
plt.pie(y.value_counts(),labels=y.value_counts().index,autopct="%1.1f%%")
plt.tight_layout()
plt.show()
```



Encoding the categorical features

```
encoder = LabelEncoder()
classes = dict()
def encodeColumns(col):
  _ =  encoder.fit_transform(col)
  classes[col.name] = encoder.classes_
  return _
car = car.apply(encodeColumns,axis="rows")
car.head()
```

	buying	maint	doors	persons	lug_boot	safety	class
0	3	3	0	0	2	1	2
1	3	3	0	0	2	2	2
2	3	3	0	0	2	0	2
3	3	3	0	0	1	1	2
4	3	3	0	0	1	2	2

Separating the features and the target

```
X,y = car.iloc[:,:-1],car.iloc[:,-1]
```

Decision Tree

Applying KFold cross-validation with 10 splits

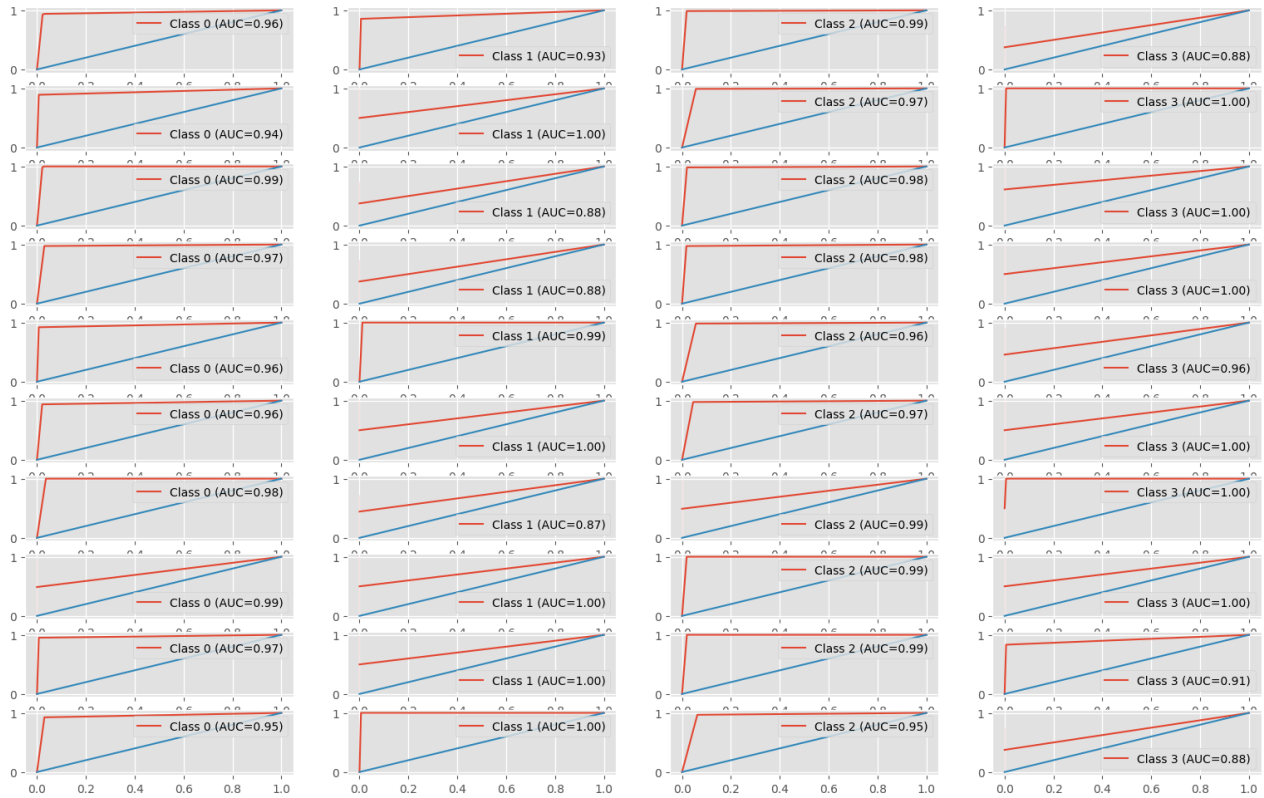
- with ROC_AUC curve

```
kf = KFold(n_splits=10,random_state=0,shuffle=True)
plt.style.use("ggplot")
plt.figure(figsize=(19.2,12))
index = 1
for train_index,test_index in kf.split(X,y):
  x_train,x_test = X.iloc[train_index],X.iloc[test_index]
  y_train,y_test = y.iloc[train_index],y.iloc[test_index]

  oversampler = SMOTE()
  x_train,y_train = oversampler.fit_resample(x_train,y_train)
  dt = DecisionTreeClassifier()
  dt_classifier = dt.fit(x_train,y_train)

  y_pred = dt_classifier.predict(x_test)
```

```
y_prob = dt.predict_proba(x_test)
y_bin = LabelBinarizer().fit_transform(y_test)
n_classes = y_bin.shape[1]
for i in range(n_classes):
  fpr, tpr, _ = roc_curve(y_bin[:, i], y_prob[:, i])
  auc_score = auc(fpr, tpr)
  plt.subplot(10,4,index)
  sns.lineplot(x=fpr, y=tpr,label=f"Class {i} (AUC={auc_score:.2f})")
  sns.lineplot(x=[0,1],y=[0,1])
  index+=1
```



Applying holdout cross-validation with

- test dataset size : 20%
- training dataset size : 80%

```
x_train,x_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=0)
```

Using Synthetic Minority Oversampling Technique (**SMOTE**) to balance the class

```
oversampler = SMOTE()
x_train,y_train = oversampler.fit_resample(x_train,y_train)
```

Applying the decision tree classifier

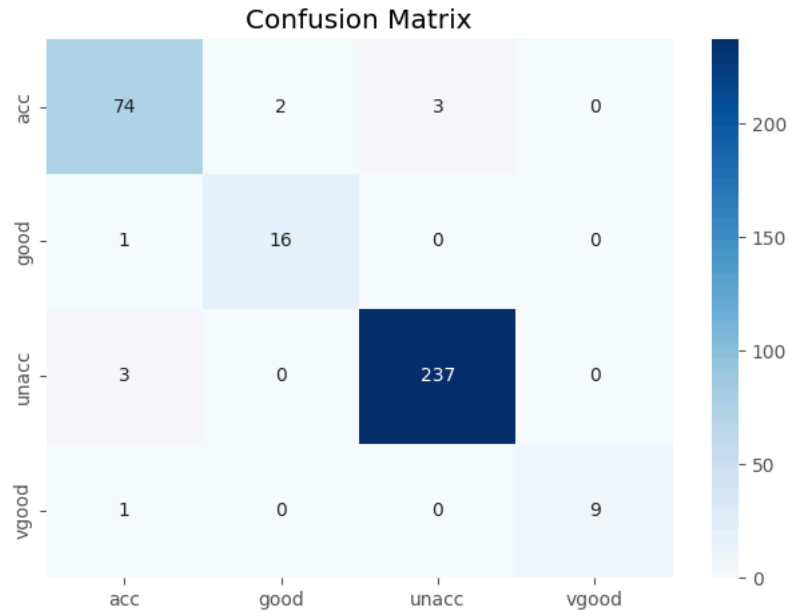
```
dt = DecisionTreeClassifier()
dt_classifier = dt.fit(x_train,y_train)
y_pred = dt_classifier.predict(x_test)
y_pred
```

```
array([0, 2, 2, 2, 2, 1, 2, 0, 1, 0, 2, 2, 0, 2, 2, 0, 2, 2, 0, 2, 1, 0,
       0, 2, 2, 2, 2, 2, 2, 2, 1, 0, 2, 0, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       3, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 2, 0, 0,
       2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 2, 2, 0, 2, 0, 0, 0, 2, 2, 2, 2, 1,
       2, 2, 2, 0, 2, 2, 2, 3, 2, 0, 0, 2, 2, 2, 0, 2, 3, 0, 2, 0, 2, 2,
       2, 2, 2, 0, 2, 1, 0, 2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 0, 1, 2, 2, 2,
       0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 0, 2, 0, 0, 0, 0, 2, 2,
```

```
       2, 0, 2, 2, 2, 2, 0, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 2,
       2, 2, 2, 2, 0, 0, 0, 2, 0, 2, 1, 2, 2, 2, 2, 0, 2, 2, 2, 0, 2, 0,
       2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 0, 2, 2,
       2, 0, 2, 3, 2, 2, 2, 3, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 1, 2, 0, 2, 2, 2, 2, 2, 1, 0, 1, 0, 0, 2, 2, 2, 0, 2, 2, 0,
       2, 2, 3, 2, 0, 2, 2, 2, 0, 2, 0, 2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 0,
       2, 3, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 2, 0, 0, 1, 1, 2, 0, 2,
       0, 2, 1, 0, 2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 0, 2, 0, 2, 2, 2, 2, 2,
       0, 2, 1, 2, 1, 2, 2, 2, 3, 2, 2, 3, 2, 0, 2, 0])
```

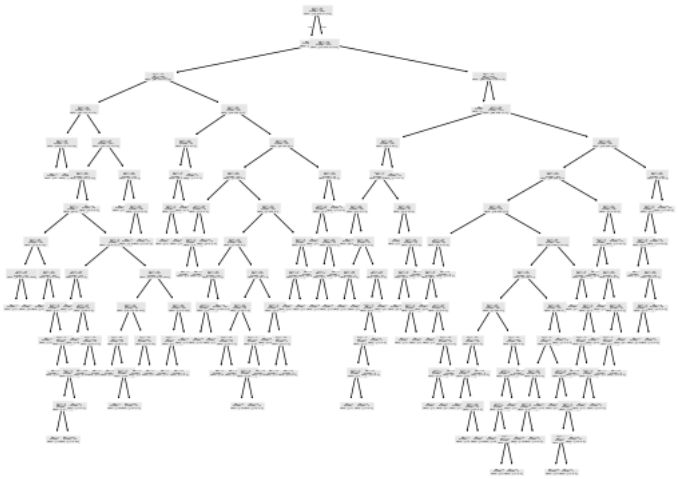
Confusion matrix for decision tree classifier

```
conf_matrix = confusion_matrix(y_test,y_pred)
sns.heatmap(conf_matrix,annot=True,fmt="d",cmap=plt.cm.Blues,xticklabels=classes['class'],yticklabels=classes['c
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()
```



Decision Tree

```
plot_tree(dt_classifier)
plt.show()
```



Decision Tree Report

```
dt_report = classification_report(y_pred=y_pred,y_true=y_test,output_dict=True)
dt_report = pd.DataFrame(dt_report).transpose()
dt_report
```

	precision	recall	f1-score	support
0	0.936709	0.936709	0.936709	79.000000
1	0.888889	0.941176	0.914286	17.000000
2	0.987500	0.987500	0.987500	240.000000
3	1.000000	0.900000	0.947368	10.000000
accuracy	0.971098	0.971098	0.971098	0.971098
macro avg	0.953274	0.941346	0.946466	346.000000
weighted avg	0.971419	0.971098	0.971146	346.000000

Naive Bayes

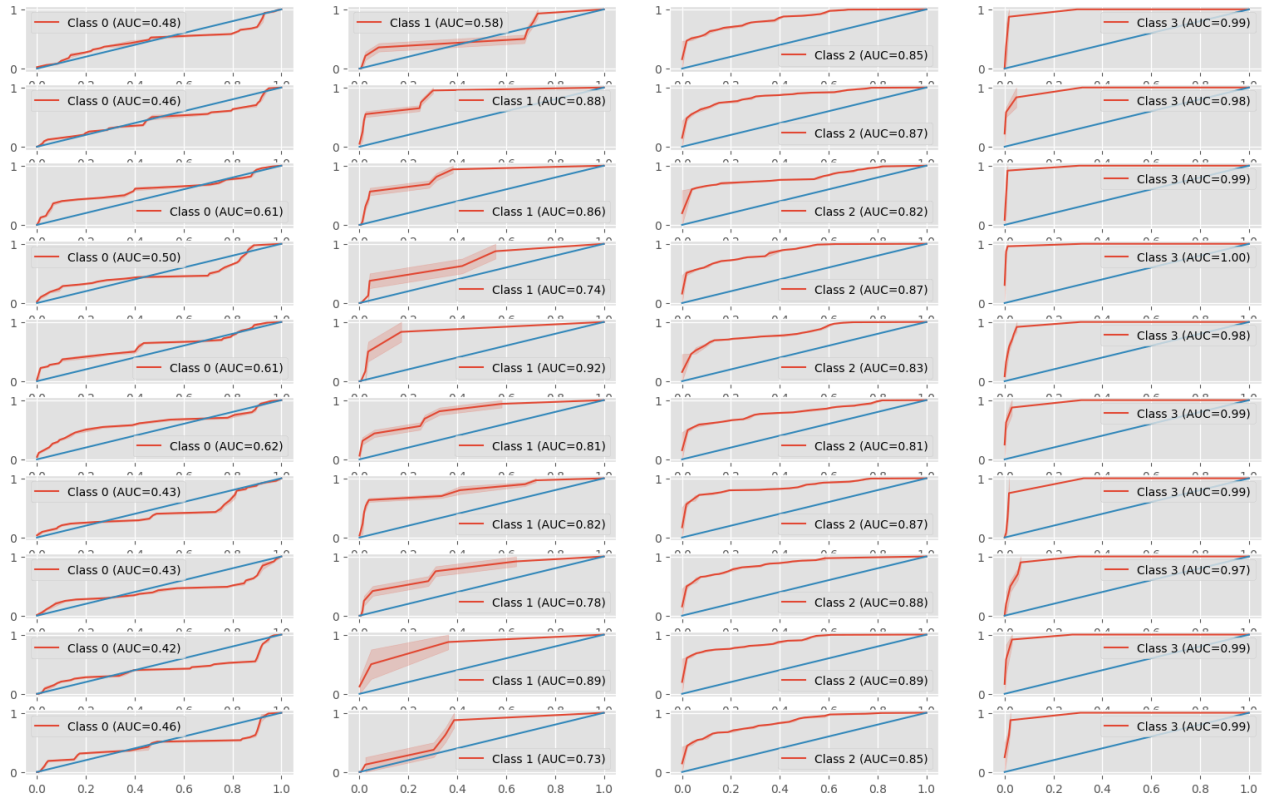
Aplying KFold cross-validation with 10 splits

- with roc-auc curve

```
kf = KFold(n_splits=10,random_state=0,shuffle=True)
plt.style.use("ggplot")
plt.figure(figsize=(19.2,12))
index = 1
for train_index,test_index in kf.split(X,y):
  x_train,x_test = X.iloc[train_index],X.iloc[test_index]
  y_train,y_test = y.iloc[train_index],y.iloc[test_index]

  oversampler = SMOTE()
  x_train,y_train = oversampler.fit_resample(x_train,y_train)
  nb = GaussianNB()
  nb_classifier = nb.fit(x_train,y_train)

  y_pred = nb_classifier.predict(x_test)
  y_prob = nb.predict_proba(x_test)
  y_bin = LabelBinarizer().fit_transform(y_test)
  n_classes = y_bin.shape[1]
  for i in range(n_classes):
    fpr, tpr, _ = roc_curve(y_bin[:, i], y_prob[:, i])
    auc_score = auc(fpr, tpr)
    plt.subplot(10,4,index)
    sns.lineplot(x=fpr, y=tpr,label=f"Class {i} (AUC={auc_score:.2f})")
    sns.lineplot(x=[0,1],y=[0,1])
    index+=1
```



Applying holdout-crossvalidation with

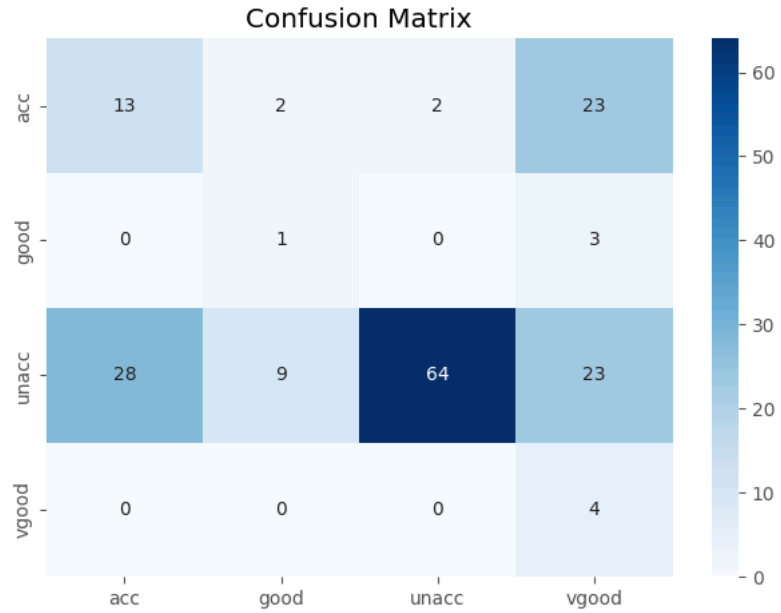
- test dataset size : 20%
- training dataset size : 80%

```
nb = GaussianNB()
nb_classifier = nb.fit(x_train,y_train)
y_pred = nb_classifier.predict(x_test)
y_pred
```

```
array([2, 3, 2, 3, 2, 2, 2, 3, 2, 2, 2, 3, 2, 2, 0, 3, 0, 0, 2, 2, 2, 3,
       3, 2, 2, 3, 2, 3, 2, 0, 0, 3, 2, 3, 2, 3, 3, 0, 0, 3, 3, 2, 2, 3,
       2, 0, 2, 0, 2, 0, 0, 2, 2, 3, 0, 0, 3, 0, 2, 3, 2, 3, 0, 0, 0, 3,
       2, 2, 0, 2, 0, 3, 3, 2, 3, 2, 2, 3, 0, 0, 0, 3, 3, 3, 3, 0, 3, 2,
       0, 3, 0, 0, 2, 3, 2, 0, 3, 2, 2, 3, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0,
       3, 0, 0, 3, 3, 0, 3, 2, 2, 0, 0, 2, 2, 3, 1, 1, 1, 1, 1, 3, 2, 2,
       3, 2, 3, 0, 3, 0, 3, 2, 0, 2, 2, 2, 2, 0, 2, 0, 3, 3, 0, 3, 0, 2,
       1, 2, 3, 1, 3, 3, 1, 3, 2, 1, 1, 2, 3, 2, 1, 1, 2, 3])
```

Confusion Matrix for naive bayes classifier

```
conf_matrix_nb = confusion_matrix(y_test,y_pred)
sns.heatmap(conf_matrix_nb,annot=True,fmt="d",cmap=plt.cm.Blues,xticklabels=classes['class'],yticklabels=classes
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()
```



Report for naive bayes classifier

```
nb_report = classification_report(y_pred=y_pred,y_true=y_test,output_dict=True)
nb_report = pd.DataFrame(nb_report).transpose()
nb_report
```

	precision	recall	f1-score	support
0	0.317073	0.325000	0.320988	40.000000
1	0.083333	0.250000	0.125000	4.000000
2	0.969697	0.516129	0.673684	124.000000
3	0.075472	1.000000	0.140351	4.000000
accuracy	0.476744	0.476744	0.476744	0.476744
macro avg	0.361394	0.522782	0.315006	172.000000
weighted avg	0.776515	0.476744	0.566499	172.000000

KNN

Applying KNN classifier with KFold cross-validation (10 splits)

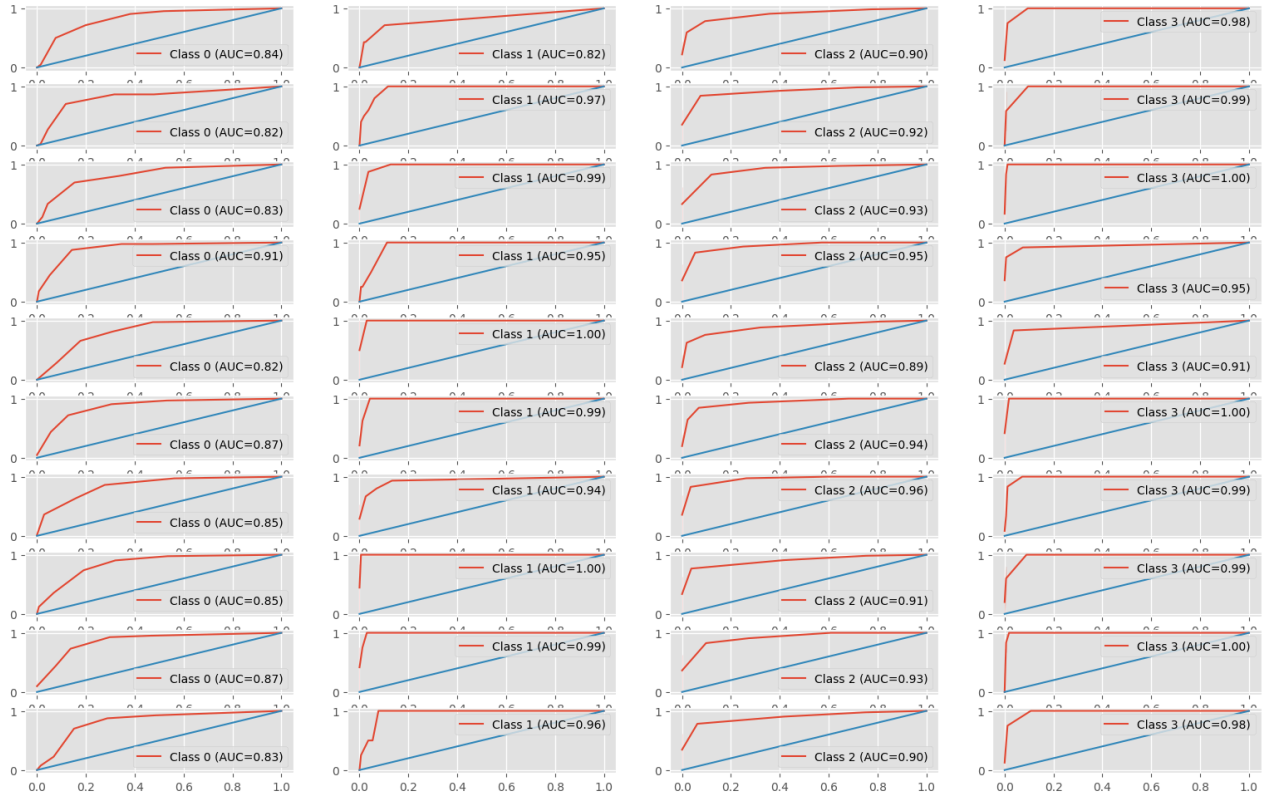
- with roc_auc curve
- k = 5 for KNN

```
kf = KFold(n_splits=10,random_state=0,shuffle=True)
plt.style.use("ggplot")
plt.figure(figsize=(19.2,12))
index = 1
for train_index,test_index in kf.split(X,y):
  x_train,x_test = X.iloc[train_index],X.iloc[test_index]
  y_train,y_test = y.iloc[train_index],y.iloc[test_index]

  oversampler = SMOTE()
  x_train,y_train = oversampler.fit_resample(x_train,y_train)
  knn = KNeighborsClassifier()
  knn_classifier = knn.fit(x_train,y_train)

  y_pred = knn_classifier.predict(x_test)
  y_prob = knn.predict_proba(x_test)
  y_bin = LabelBinarizer().fit_transform(y_test)
  n_classes = y_bin.shape[1]
  for i in range(n_classes):
    fpr, tpr, _ = roc_curve(y_bin[:, i], y_prob[:, i])
    auc_score = auc(fpr, tpr)
    plt.subplot(10,4,index)
```

```
    sns.lineplot(x=fpr, y=tpr,label=f"Class {i} (AUC={auc_score:.2f})")
    sns.lineplot(x=[0,1],y=[0,1])
    index+=1
```



Applyig KNN classifier with holdout-crossvalidation and K=5

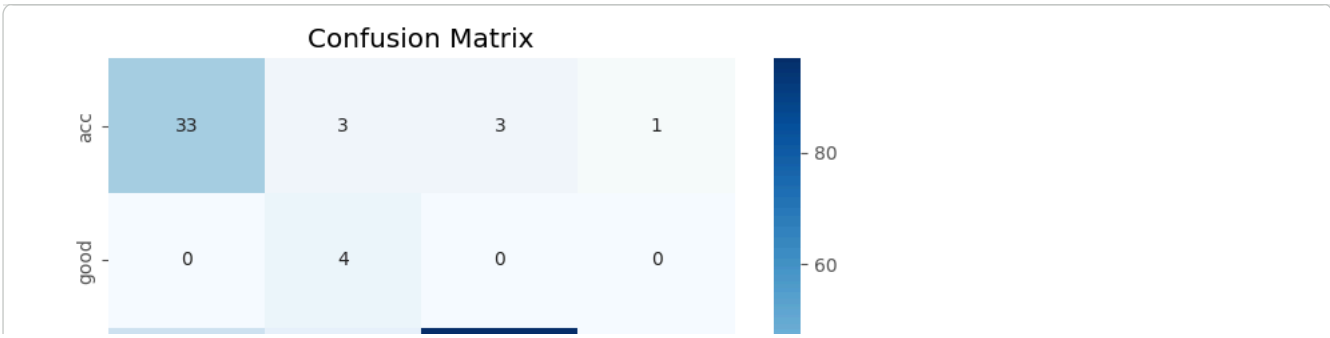
- test dataset size : 80%
- training dataset size : 20%

```
knn = KNeighborsClassifier()
knn_classifier = knn.fit(x_train,y_train)
y_pred = knn_classifier.predict(x_test)
y_pred
```

```
array([2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 0, 2, 2, 2, 0, 2, 0, 2, 0, 2, 0, 0, 2, 2, 3, 2, 0, 0, 0,
       2, 0, 2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 0, 0, 2, 2, 2, 0, 2, 0, 2, 0,
       2, 2, 2, 2, 0, 0, 2, 2, 0, 2, 2, 0, 2, 2, 0, 0, 0, 2, 0, 0, 0, 2,
       0, 2, 2, 0, 2, 0, 2, 0, 0, 2, 0, 0, 2, 2, 2, 2, 2, 2, 0, 0, 2, 1,
       0, 2, 0, 2, 0, 0, 0, 0, 2, 2, 2, 2, 0, 0, 0, 0, 1, 2, 0, 1, 2, 2,
       2, 0, 1, 1, 0, 0, 0, 2, 1, 0, 2, 2, 2, 0, 2, 2, 2, 2, 0, 1, 0, 2,
       1, 2, 2, 2, 3, 2, 2, 1, 2, 1, 1, 2, 3, 1, 1, 2, 2, 1])
```

Confusion Matrix for KNN

```
conf_matrix_knn = confusion_matrix(y_test,y_pred)
sns.heatmap(conf_matrix_knn,annot=True,fmt="d",cmap=plt.cm.Blues,xticklabels=classes['class'],yticklabels=classe
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()
```


Confusion Matrix
acc
good
33
3
3
1
0
4
0
0
80
60